

[CS230] Sequence Models: The Transformer Architecture

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-9. Deep Learning Fundamentals & CNNs - *Completed*

Chapter 10. Sequence Models (Current Part)

- 10.1-10.11 RNN, LSTM, Self-Attention - *Completed*
- **10.12 The Transformer Architecture**
 - * Positional Encoding: Giving Order to Sets
 - * Encoder Block: Deep Understanding
 - * Decoder Block: Sequential Generation
 - * BERT vs GPT: Architectural Philosophy

지난 시간 복습 및 연결

우리는 지난 시간에 Self-Attention이라는 트랜스포머의 핵심 엔진을 배웠습니다. 오늘은 이 엔진들을 조립해서 어떻게 BERT나 GPT 같은 거대 모델의 뼈대가 되는 **트랜스포머 아키텍처 (Transformer Architecture)**가 완성되는지 해부해 보겠습니다. 트랜스포머는 크게 정보를 읽어 들이는 **인코더(Encoder)**와 정보를 생성하는 **디코더(Decoder)** 두 부분으로 나뉩니다.

1 Unit Overview

□ 핵심 목표

이 단원은 현대 AI 시스템의 설계도인 트랜스포머의 구조를 완벽히 이해합니다.

- **전체 구조:** 인코더와 디코더의 연결 메커니즘을 파악합니다.
- **순서 인식:** RNN 없이 순서를 파악하는 **Positional Encoding**을 배웁니다.
- **안정성:** 깊은 층을 쌓기 위한 **Add & Norm** (잔차 연결 및 정규화) 장치를 확인합니다.
- **철학의 차이:** 인코더 기반의 **BERT**와 디코더 기반의 **GPT** 차이를 이해합니다.

2 Core Components: 설계도 해부

2.1 1. Positional Encoding (위치 정보 주입)

트랜스포머는 단어를 한꺼번에 병렬로 입력받기 때문에, "I love you"와 "You love me"를 구분하지 못합니다.

□ 위치 신호 주입 (핵심 원리)

각 단어 벡터에 고유한 위치 값을 더해줍니다. 주기 함수인 Sine과 Cosine을 활용하여 고차원 공간에서 단어의 상대적/절대적 위치를 입힙니다.

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

2.2 2. Encoder Block (인코더: 이해의 영역)

인코더는 입력 문장을 통째로 보고 문맥을 파악합니다.

- **Multi-Head Self-Attention:** 단어 사이의 유기적 관계를 계산합니다.
- **Add & Norm:** 잔차 연결로 기울기 소실을 막고 Layer Norm으로 학습을 돕습니다.
- **BERT:** 이 인코더를 쌓아 만든 모델로, 문맥을 양방향(Bi-directional)으로 이해합니다.

2.3 3. Decoder Block (디코더: 생성의 영역)

디코더는 분석된 정보를 바탕으로 단어를 하나씩 생성합니다.

- **Masked Self-Attention:** 단어 생성 시 미래 단어를 미리 보고 정답을 유추하지 못하도록 마스킹을 적용합니다.
- **Encoder-Decoder Attention:** 단어를 뱉을 때마다 인코더가 분석한 원문 정보를 다시 참고합니다.
- **GPT:** 이 디코더를 쌓아 만든 모델로, 다음 단어 예측(Auto-regressive)에 특화되어 있습니다.

3 Deep Dive: BERT vs GPT

같은 트랜스포머 뿌리를 두지만, 부품 선택에 따라 성격이 완전히 달라집니다.

특징	BERT	GPT
기본 구조	트랜스포머 인코더	트랜스포머 디코더
학습 방향	양방향 (Bi-directional)	단방향 (Left-to-Right)
주요 목적	문장 분류, 질의응답	문장 생성, 대화, 창작
비유	빈칸 채우기 잘하는 모범생	이야기를 지어내는 소설가

4 Implementation: Hugging Face Transformers

현대 개발자들은 사전 학습된 가중치를 불러와서 미세 조정(Fine-tuning)하여 사용합니다.

```
1 from transformers import BertModel, GPT2Model
2
3 # BERT: 의미 추출 및 문장 분류 시
4 bert = BertModel.from_pretrained('bert-base-uncased')
5
6 # GPT: 텍스트 생성 및 챗봇 구현 시
7 gpt = GPT2Model.from_pretrained('gpt2')
```

Listing 1: Loading Pre-trained Transformers

□ Summary & Next Step

1. **Transformer:** RNN을 대체한 완벽한 병렬 처리 아키텍처.
2. **Positional Encoding:** 어텐션에 '순서'라는 생명력을 불어넣는 장치.
3. **Functional Split:** 이해를 원하면 인코더(BERT), 생성을 원하면 디코더(GPT).

이제 여러분은 현대 인공지능이 인간의 언어를 어떻게 처리하는지 그 밑바닥 설계도를 보셨습니다. 이 구조는 이제 언어를 넘어 이미지(ViT), 음성(Whisper) 등으로 무한히 확장되고 있습니다.

□ 생각해볼 거리

마스킹(Masking)이 없다면 디코더는 왜 학습이 불가능할까요? 잔차 연결(Residual Connection)이 깊은 트랜스포머 층에서 왜 필수적일까요? 궁금한 점이 있다면 언제든지 질문해주세요!